Relatório de Auditoria de Segurança — CustomerAfterSales

Data: 15/09/2026

Escopo: repositório PettaDev/CustomerAfterSales, commit main 6426c528bd3a2b1ef4cc86790a1a86b8cf338990

Abrangência: frontend React/Vite, API Express, persistência PostgreSQL/SQLite, storage, Support Bridge, autenticação, CI/CD, deploy Vercel, histórico Git e bundle de frontend.

Nota metodológica

A auditoria foi adaptada à stack detectada. 'Banco sem tranca' foi mapeado para o mecanismo real de isolamento por token de caso + sessão staff, pois o projeto não usa Supabase/RLS nem modelo multi-tenant por organização. 'Permissão no navegador' foi cruzada entre gates React e os endpoints Express. IDOR foi percorrido handler por handler. Segredos foram verificados no código/configuração, histórico Git e bundle por formatos de credencial de alta confiança. XSS foi revisado procurando sinks de HTML/JS e a origem de URLs renderizadas.

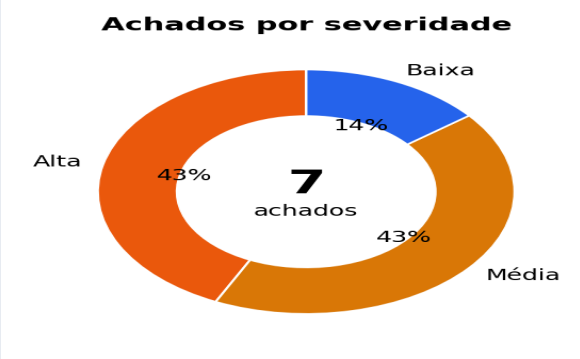
Limite metodológico: esta é uma auditoria de código e testes automatizados do repositório. Não substitui pentest externo autenticado nem análise de infraestrutura de terceiros. Achados condicionais são explicitamente marcados.

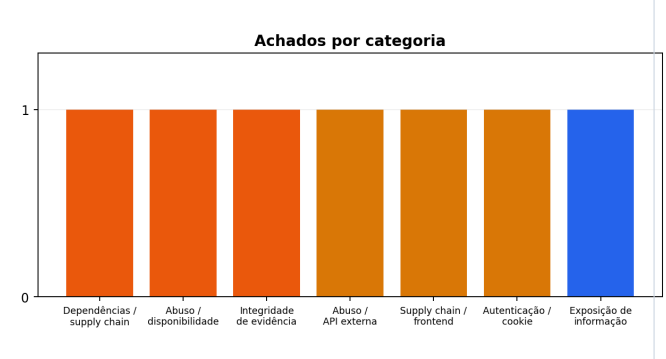
1. Stack detectada

Camada	Tecnologia detectada
Linguagens	TypeScript/TSX no frontend; JavaScript ESM (.mjs) no backend e bridge; Node.js >=22
Frontend	React 19, Vite 7, React Router DOM, i18next, Framer Motion, Lucide, Tailwind/PostCSS
Backend	Express 5 + Zod
Banco	PostgreSQL (Neon em produção) via pacote postgres; fallback local node:sqlite. Sem ORM/query builder.
Auth staff	STAFF_EMAIL + hash scrypt; sessão aleatória de 32 bytes em cookie tfae, token armazenado por SHA-256
Auth cliente	Token aleatório de 32 bytes por caso; apenas accessHash é persistido; bearer token acompanha operações do caso
Storage	Vercel Blob privado ou S3-compatible via AWS SDK; URLs assinadas; disco local no desenvolvimento
Bridge	Express local em 127.0.0.1:43127 com Host/Origin + bearer aleatório
Deploy/CI	vercel.json + GitHub Actions; não há Dockerfile, Helm ou Terraform no repositório auditado

2. Resumo executivo

Foram confirmados 7 achados acionáveis: 3 de severidade alta, 3 média e 1 baixa. Nenhum achado crítico foi verificado. Nas cinco categorias obrigatórias, não foi confirmada falha de isolamento/IDOR, gate privilegiado apenas no browser, segredo hardcoded ou XSS direto. Os riscos centrais encontrados são abuso de endpoints públicos, integridade de evidências após o complete e dependência de JavaScript ADB de CDN em runtime.





Severidade	Quantidade
Crítica	0
Alta	3
Média	3
Baixa	1

3. Mapeamento das cinco categorias obrigatórias

1. Banco sem tranca

Não há RLS/Supabase nem tenants por organização. O isolamento do cliente é token por caso em access(); staff é um workspace global. A listagem global só existe para staff. Resultado: nenhum bypass de isolamento confirmado.

2. Permissão definida no navegador

O frontend usa staff && para controles privilegiados, mas o backend aplica requireStaff em listagem e PATCH. Resultado: nenhum gate somente visual confirmado.

3. IDOR

Todos os handlers Express por ID foram percorridos: case, evidence, upload, complete, download e session. Todos verificam case token/staff ou token específico de upload. Bridge possui middleware global antes das rotas. Resultado: nenhum IDOR confirmado.

4. Chaves expostas

.env é ignorado, .env.example contém placeholders vazios e não há histórico de .env. O scanner de formatos de credencial cobre histórico Git e dist/. Resultado: nenhum segredo hardcoded confirmado.

5. Inputs sem tratamento (XSS)

Não foram encontrados dangerouslySetInnerHTML, innerHTML, eval ou new Function. Dados de cliente/IA são JSX escapado. URLs do guia vêm de JSON local versionado. Não há biblioteca de sanitização, mas também não há sink de HTML bruto que a exija hoje. Resultado: nenhum XSS direto confirmado.

4. Pontos fortes verificados

● Isolamento do cliente por token de caso

Evidência: server/app.mjs:31-55, 245-258, 289-320, 346-366

O backend não confia no ID do caso sozinho: access() exige o hash do token do caso ou uma sessão staff válida. Evidências e sessões derivam o caseId e voltam a aplicar access().

● Gate privilegiado no servidor

Evidência: server/app.mjs:233-243, 260-287; src/portal/Cases.tsx:194-250

A UI esconde controles de status/prioridade/owner/note com staff &&, mas o endpoint PATCH correspondente também usa requireStaff. A listagem global de casos também é staff-only.

● Queries parametrizadas

Evidência: server/store.mjs:55-186

PostgreSQL usa tagged templates do pacote postgres e SQLite usa placeholders. Não foi encontrado SQL construído por concatenação de input do usuário.

● Sem IDOR verificado nos handlers por ID

Evidência: server/app.mjs; bridge/server.mjs

Todos os handlers de case/evidence/session por ID possuem posse/autorização equivalente. O bridge aplica middleware global de Host/Origin/Bearer antes das rotas.

● Proteção de segredos no repositório atual

Evidência: .gitignore; .env.example; docs/production-setup.md

.env é ignorado; .env.example contém placeholders vazios; documentação orienta variáveis de ambiente. Scanner dedicado verifica histórico e bundle por formatos de credencial de alta confiança.

● Renderização React sem HTML bruto

Evidência: src/portal/Cases.tsx; src/components/LogAssistant.tsx; src/components/GuideBlockRenderer.tsx

Dados de cliente e respostas de IA são interpolados em JSX e escapados pelo React. Não foram encontrados dangerouslySetInnerHTML, innerHTML, eval ou new Function.

● URLs do guia não são input de cliente

Evidência: src/guides/index.ts; src/components/GuideBlockRenderer.tsx

href/src/iframe do guia vêm de JSON local importado no build por import.meta.glob, não de campos do caso ou mensagens do usuário.

● Storage privado e URLs curtas

Evidência: server/storage.mjs:34-113

Vercel Blob usa access=private e downloads expiram em 2 minutos; S3 usa presigned URLs. O backend verifica o tamanho no fluxo de conclusão.

● Sessão staff robusta no Vercel

Evidência: server/security.mjs; server/app.mjs:145-176

Senha usa scrypt com salt; tokens têm 32 bytes aleatórios, são armazenados por hash, comparação é timing-safe e o cookie atual é httpOnly, SameSite=Strict e Secure no Vercel.

● Bridge local com fronteira forte

Evidência: bridge/server.mjs:1-35; bridge/core.mjs

Bridge escuta somente 127.0.0.1, valida Host/Origin e exige bearer aleatório. ADB usa execFile/spawn com arrays; serial é validado; PIDs são verificados antes de kill.

5. Pontos fracos e achados detalhados

Somente itens confirmados no código real são listados abaixo. Condições necessárias à exploração aparecem em cada achado.

F-00 - Dependências de produção possuem vulnerabilidades conhecidas, incluindo undici de severidade alta

Alta	package.json:22,24; package-lock.json

O workflow dedicado executou npm audit --omit=dev contra o lockfile real e encontrou 5 vulnerabilidades de produção (2 baixas, 2 moderadas e 1 alta). O relatório identifica undici <=6.27.0 com advisories de exaustão de recursos, request/response smuggling e outros problemas, além de @ai-sdk/provider-utils vulnerável a consumo descontrolado de recursos. A árvore vulnerável é alcançada pelas versões atuais de ai 6.0.0 e @ai-sdk/google 3.0.103.

Por que é explorável

Os pacotes vulneráveis estão instalados como dependências de produção. O assistente usa a AI SDK quando a chave Google está configurada. A explorabilidade exata varia por advisory e pelo caminho de código utilizado, mas o lockfile de produção satisfaz as faixas afetadas confirmadas pelo npm audit; portanto a cadeia deve ser atualizada antes de ampliar exposição.

Condição de explorabilidade

Os pacotes afetados estão presentes no lockfile de produção; exploração específica depende do advisory/caminho utilizado. O fluxo de IA só é ativo quando a chave do provedor está configurada.

Trecho relevante

```
"@ai-sdk/google": "3.0.103",
"ai": "6.0.0"

# npm audit --omit=dev
5 vulnerabilities (2 low, 2 moderate, 1 high)
undici <=6.27.0 - Severity: high
```

F-01 - Criação pública de casos e emissão de uploads de até 1 GiB sem quota de abuso

Alta	server/app.mjs:215-230; 289-320

POST /api/cases é público e entrega um token válido para o novo caso. Com esse token, POST /api/cases/:id/evidence pode emitir repetidamente URLs assinadas para arquivos de até 1 GiB. Não existe limite de casos por IP, quantidade de evidências por caso, orçamento total de bytes por caso nem reserva de quota antes da emissão da URL.

Por que é explorável

Um cliente automatizado pode omitir o header Origin, criar muitos casos e solicitar uploads grandes em sequência. Isso permite consumir banco, operações de storage e capacidade/cota do Blob/S3. No plano gratuito, o impacto principal é indisponibilidade por esgotamento de quota; em plano pago, também pode gerar custo.

Condição de explorabilidade

Explorável quando a API estiver publicamente acessível, como no deployment Vercel atual.

Trecho relevante

```
app.post("/api/cases", async (req, res) => { ...
  res.status(201).json({ case: safeCase(c), accessToken: t });
});

app.post("/api/cases/:id/evidence", async (req, res) => {
  const c = await access(req, req.params.id);
  ...
  size: z.number().int().positive().max(1024 * 1024 * 1024),
  ...
  const url = await uploadURL(e);
});
```

F-02 - URL de upload continua reutilizável e permite sobrescrever evidência após a conclusão

Alta	server/storage.mjs:34-75; server/app.mjs:346-353

A URL assinada usa um caminho/chave determinístico. No Vercel Blob, allowOverwrite está explicitamente true; a URL fica válida por 15 minutos. No backend, o fluxo /complete verifica o tamanho uma única vez e marca a evidência como uploaded=true, mas não revoga a URL já emitida. Em S3, o PutObject assinado usa a mesma Key e também pode ser reutilizado durante a validade.

Por que é explorável

O próprio titular legítimo do token do caso pode concluir uma evidência e, enquanto a URL PUT ainda estiver válida, reenviar conteúdo para a mesma chave. A base continua indicando que o arquivo já foi validado, quebrando a imutabilidade e a cadeia de custódia do material analisado pelo TFAE.

Condicão de explorabilidade

Explorável enquanto a URL assinada original permanecer válida; depende de posse legítima do token do caso/URL de upload.

Trecho relevante

```
const { presignedUrl } = await presignUrl(signedToken, {
  pathname,
  operation,
  access: "private",
  ...
  addRandomSuffix: false,
  allowOverwrite: true,
});

app.post("/api/evidence/:id/complete", async (req, res) => {
  ...
  await verifyUpload(e);
  await db.put("evidence", e.id, { ...rest, uploaded: true });
});
```

F-03 - Endpoint público de IA sem rate limit pode consumir quota do provedor

Média	server/app.mjs:72-74; server/assistant.mjs:3-31

POST /api/chat não exige autenticação nem aplica rate limiting próprio. Quando GOOGLE_GENERATIVE_AI_API_KEY está configurada, cada chamada válida chega ao generateText do provedor Google, com até 700 tokens de saída e timeout de 25 s.

Por que é explorável

Um script pode chamar o endpoint diretamente sem Origin e gerar carga/concorrência e consumo de quota. A proteção de Origin é um controle de navegador/CSRF, não um mecanismo de autenticação para clientes HTTP arbitrários.

Condicão de explorabilidade

Somente explorável quando GOOGLE_GENERATIVE_AI_API_KEY estiver configurada e o assistente estiver habilitado.

Trecho relevante

```
app.post("/api/chat", async (req, res) =>
  res.json({ reply: await reply(req.body) }),
);

const { text } = await generateText({
  model: google(process.env.GEMINI_MODEL || "gemini-3.5-flash-lite"),
  maxOutputTokens: 700,
  abortSignal: AbortSignal.timeout(25000),
  ...
});
```

F-04 - Bibliotecas ADB sensíveis são executadas por import remoto de CDN em runtime

Média	src/portal/BrowserCapture.tsx:42; 53-55

O fluxo WebUSB/WebADB usa import dinâmico de três módulos JavaScript hospedados no jsDelivr. As versões estão fixadas, mas os bytes executados não fazem parte do bundle/lockfile entregue pela aplicação e o import() remoto não utiliza SRI.

Por que é explorável

Comprometimento da cadeia npm/CDN, resolução maliciosa ou entrega indevida daquele recurso faria JavaScript externo executar na origem do Aftercare, justamente na tela que possui acesso a WebUSB/ADB e dados do caso.

Condicão de explorabilidade

Exige comprometimento/falha na cadeia externa; não há evidência de comprometimento atual.

Trecho relevante

```
const remoteImport = (url: string): Promise<any> => import(/* @vite-ignore */ url);
...
remoteImport("https://cdn.jsdelivr.net/npm/@yume-chan/adb@2.1.0/+esm"),
remoteImport("https://cdn.jsdelivr.net/npm/@yume-chan/adb-daemon-webusb@2.1.0/+esm"),
remoteImport("https://cdn.jsdelivr.net/npm/@yume-chan/adb-credential-web@2.1.0/+esm"),
```

F-05 - Flag Secure do cookie TFAE depende de VERCEL, não do modo de produção/HTTPS

Média | server/app.mjs:164-171

O cookie de sessão TFAE usa httpOnly e SameSite=Strict, porém a flag Secure é definida por !!process.env.VERCEL. O projeto também suporta execução própria via npm start/server/index.mjs, onde uma implantação HTTPS pode estar em produção sem a variável VERCEL.

Por que é explorável

Em uma implantação self-hosted HTTPS com VERCEL ausente, o navegador aceitará o cookie sem Secure. Se a mesma origem puder ser alcançada por HTTP, o cookie pode ser transmitido sem TLS e expor a sessão.

Condição de explorabilidade

Não é explorável no deployment Vercel atual, pois VERCEL é definido. Risco condicional para self-hosting/ambientes futuros.

Trecho relevante

```
.cookie("tfae", t, {
  httpOnly: true,
  secure: !!process.env.VERCEL,
  sameSite: "strict",
  maxAge: 8 * 3600000,
  path: "/",
})
```

F-06 - Health check público expõe detalhes desnecessários de infraestrutura

Baixa | server/app.mjs:79-109

GET /api/health retorna publicamente estado do banco, latência do banco, estado do storage, presença de autenticação staff e tempo de resposta. O erro textual do banco é ocultado em production, o que é correto, mas os demais sinais permanecem públicos.

Por que é explorável

Um atacante pode usar o endpoint para reconhecimento e para detectar períodos de cold start/degradação, facilitando a escolha do momento de abuso. O impacto isolado é baixo porque não há segredo, credencial nem dado de cliente no retorno.

Condição de explorabilidade

Explorável sempre que /api/health estiver público.

Trecho relevante

```
res.status(ok ? 200 : 503).json({
  ok,
  database,
  databaseLatencyMs,
  storage,
  staff: !!process.env.STAFF_PASSWORD_HASH,
  responseTimeMs: Date.now() - started,
  ...
});
```

6. Tabela consolidada de achados

Severidade	Arquivo:linha	Descrição
Alta	package.json:22,24; package-lock.json	Dependências de produção possuem vulnerabilidades conhecidas, incluindo undici de severidade alta
Alta	server/app.mjs:215-230; 289-320	Criação pública de casos e emissão de uploads de até 1 GiB sem quota de abuso
Alta	server/storage.mjs:34-75; server/app.mjs:346-353	URL de upload continua reutilizável e permite sobrescrever evidência após a conclusão
Média	server/app.mjs:72-74; server/assistant.mjs:3-31	Endpoint público de IA sem rate limit pode consumir quota do provedor
Média	src/portal/BrowserCapture.tsx:42; 53-55	Bibliotecas ADB sensíveis são executadas por import remoto de CDN em runtime
Média	server/app.mjs:164-171	Flag Secure do cookie TFAE depende de VERCEL, não do modo de produção/HTTPS
Baixa	server/app.mjs:79-109	Health check público expõe detalhes desnecessários de infraestrutura

7. Testes gerais e evidências de cobertura

Verificação	Resultado
CI main no commit auditado	SUCCESS
npm test no main	12/12 testes passaram; 0 falhas
Build TypeScript + Vite	SUCCESS
Lint	64 erros e 2 warnings ainda existem; etapa está continue-on-error e portanto não bloqueia release
Teste adicional de auditoria	Executado no workflow: 13/13 testes passaram; inclui autorização por objeto, cross-case, staff e Origin
npm audit produção	Encontrou 5 vulnerabilidades no lockfile de produção: 2 baixas, 2 moderadas e 1 alta; a alta envolve undici <=6.27.0
Secret scan	PASS: nenhum formato de credencial de alta confiança encontrado no histórico Git completo ou em dist/

Observação importante: o lint atual contém dívida técnica pré-existente e, por estar configurado com continue-on-error, um workflow verde não significa lint limpo. Recomenda-se tornar lint bloqueante após corrigir a dívida.

8. Recomendações priorizadas

P1 - Atualizar dependências vulneráveis de produção

Atualizar AI SDK/Google SDK e a cadeia undici/provider-utils para versões corrigidas; validar npm audit, testes e comportamento do assistente.

P2 - Bloquear abuso de storage/DB

Rate limit, quotas por caso/IP, soma de bytes, reserva de quota e testes 429/limite.

P3 - Garantir imutabilidade da evidência

Chaves/versões únicas, sem overwrite após complete, persistir hash/ETag/version e validar identidade.

P4 - Trazer Tango/WebADB para o bundle

Remover import remoto do jsDelivr e fixar dependências no package-lock; depois adotar CSP mais restritiva.

P5 - Rate limit do assistente

Aplicar orçamento e concorrência; desabilitar endpoint quando a feature não estiver ativa.

P6 - Endurecer cookie para self-hosting

Secure por política explícita de produção e teste de configuração.

P7 - Minimizar health público

Separar liveness público de readiness detalhado protegido.

P8 - Pagar dívida de lint

Corrigir 64 erros/2 warnings e remover continue-on-error para que regressões futuras bloqueiem merge.

9. ISSUES PARA O GITHUB

Os blocos abaixo estão em Markdown completo, prontos para copiar e colar. Achados de abuso relacionados foram agrupados para reduzir spam de issues.

--- ISSUE 1 ---

```
# [Segurança] Atualizar dependências de produção vulneráveis da AI SDK/undici
Labels sugeridas: security, severity:high

## Descrição do problema
O `npm audit --omit=dev` executado contra o lockfile real encontrou 5 vulnerabilidades de produção (2 baixas, 2 moderadas e 1 alta). A falha alta afeta `undici <=6.27.0`; também há `@ai-sdk/provider-utils` vulnerável a consumo descontrolado de recursos. As versões atuais `ai@6.0.0` e `@ai-sdk/google@3.0.103` alcançam a árvore afetada.

## Evidência
- `package.json:22,24`
- `package-lock.json`
- Workflow `Security Audit`: `npm audit --omit=dev`

```json
"@ai-sdk/google": "3.0.103",
"ai": "6.0.0"
```

O audit reporta `undici <=6.27.0` como severidade alta e a cadeia `@ai-sdk/provider-utils` como moderada.

Impacto
Dependendo do advisory e do caminho exercitado, a aplicação pode ficar exposta a exaustão de recursos/DoS e outras classes reportadas para a versão transitiva do cliente HTTP. O risco aumenta quando o assistente está habilitado e recebe tráfego público.

Sugestão de correção
Atualizar `@ai-sdk/google`, `ai` e dependências transitivas para versões fora das faixas vulneráveis, evitando aplicar `npm audit fix --force` diretamente em produção sem revisar mudanças de versão. Rodar regressão do assistente, testes e novo `npm audit`.

Critérios de aceite
- [ ] `npm audit --omit=dev` não reporta vulnerabilidade alta na árvore de produção.
- [ ] `undici` resolvido está fora da faixa vulnerável indicada pelo advisory.
- [ ] `@ai-sdk/provider-utils` está em versão corrigida.
- [ ] Fluxo `/api/chat` continua funcional quando habilitado.
- [ ] `npm test` e `npm run build` passam após a atualização.
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
# [Segurança] Adicionar proteção de abuso, rate limit e quotas aos endpoints públicos
Labels sugeridas: security, severity:high

## Descrição do problema
Os endpoints públicos de criação de casos e registro de evidências permitem que qualquer cliente crie um caso, receba um token válido e emita URLs assinadas para arquivos de até 1 GiB, sem quota por IP/caso ou orçamento total de bytes. O endpoint `/api/chat`, quando a chave do Gemini está configurada, também não possui rate limit.

Isso permite automação para esgotar quota de banco/storage e, quando o assistente estiver habilitado, consumir quota/custo do provedor de IA.

## Evidência
- `server/app.mjs:215-230`: criação pública de caso e retorno de `accessToken`.
- `server/app.mjs:289-320`: registro de evidência com tamanho de até 1 GiB e emissão de URL assinada.
- `server/app.mjs:72-74` + `server/assistant.mjs:3-31`: `/api/chat` chama o modelo sem rate limit.

Trecho:
```js
app.post("/api/cases", async (req, res) => {
...
```
```

```
res.status(201).json({ case: safeCase(c), accessToken: t });
});
size: z.number().int().positive().max(1024 * 1024 * 1024)
```

Impacto
- Esgotamento de quota e indisponibilidade do piloto.
- Crescimento artificial do banco.
- Consumo de operações/armazenamento Blob/S3.
- Consumo de quota/custo de Gemini quando habilitado.

Sugestão de correção
Aplicar rate limiting server-side/edge por IP e por sessão; limitar casos criados por janela; definir quantidade e soma máxima de bytes por caso; reservar quota antes de gerar URL assinada; adicionar orçamento/concurrency cap para `/api/chat`; considerar challenge anti-bot em criação anônima.

Critérios de aceite
- [ ] `/api/cases` possui limite verificável por IP/janela.
- [ ] `/api/cases/:id/evidence` recusa quando quantidade ou soma de bytes do caso exceder a quota.
- [ ] A geração da URL assinada ocorre somente após reserva de quota.
- [ ] `/api/chat` possui rate limit e limite de concorrência quando habilitado.
- [ ] Testes automatizados cobrem respostas 429/413/limite de quota.
- [ ] Métricas/logs permitem identificar abuso sem registrar tokens/segredos.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Tornar evidências imutáveis após a conclusão do upload
Labels sugeridas: security, severity:high

Descrição do problema
A URL de upload é emitida para uma chave determinística e permanece válida depois que `/api/evidence/:id/complete` marca a evidência como concluída. No Vercel Blob, `allowOverwrite: true` está explícito; no S3, o mesmo `PutObject` assinado pode sobrescrever a mesma Key durante sua validade.

Assim, uma URL PUT ainda válida pode substituir os bytes após a verificação do backend, enquanto o banco continua marcando o objeto como `uploaded: true`.

Evidência
- `server/storage.mjs:34-75`
- `server/app.mjs:346-353`

Trecho:
```js
addRandomSuffix: false,
allowOverwrite: true,
...
await verifyUpload(e);
await db.put("evidence", e.id, { ...rest, uploaded: true });
```

Impacto
Quebra de integridade/cadeia de custódia: o arquivo revisado pelo TFAE pode não ser o mesmo arquivo verificado no momento da conclusão.

Sugestão de correção
Usar chave única e imutável por tentativa/versão; desabilitar overwrite quando suportado; reduzir TTL; gravar hash criptográfico e/ou versão/ETag no complete; validar essa identidade no download; para S3/R2 considerar versionamento/condições de escrita.

Critérios de aceite
- [ ] Uma URL antiga não consegue alterar uma evidência concluída.
- [ ] Cada upload possui chave/versão única.
- [ ] O backend persiste hash/ETag/version id no complete.
- [ ] Download valida ou referencia exatamente a versão concluída.
- [ ] Teste automatizado tenta reusar a URL após complete e confirma que o conteúdo final não muda.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Remover imports ADB de CDN do caminho WebUSB
Labels sugeridas: security, severity:medium

Descrição do problema
O BrowserCapture executa em runtime três módulos ADB vindos do jsDelivr por `import()` remoto. As versões estão fixas, porém os bytes executados não são parte do artefato Vite/lockfile e não há SRI aplicável ao import dinâmico.

Uma falha/comprometimento da cadeia npm/CDN permitiria executar JavaScript de terceiro na origem do Aftercare, na mesma tela que controla WebUSB/ADB.

Evidência
`src/portal/BrowserCapture.tsx:42,53-55`

```ts
const remoteImport = (url: string): Promise<any> => import(/* @vite-ignore */ url);
remoteImport("https://cdn.jsdelivr.net/npm/@yume-chan/adb@2.1.0/+esm")
```

Impacto
Comprometimento de supply chain com acesso ao contexto do portal, dados disponíveis no browser e operações WebUSB autorizadas pelo usuário.

Sugestão de correção
Adicionar os pacotes `@yume-chan/*` ao `package.json`, travar hashes no `package-lock.json` e empacotar com Vite. Aplicar CSP restritiva depois de remover a dependência de script remoto.

Critérios de aceite
- [ ] Nenhum `import()` de código executável ADB aponta para CDN.
- [ ] Dependências ADB estão no `package.json` e `package-lock.json`.
- [ ] Build offline reproduz o bundle sem baixar JavaScript em runtime.
- [ ] Fluxo WebUSB continua funcionando nos dispositivos MTK/QTI validados.
- [ ] CSP pode bloquear `script-src` de terceiros sem quebrar a coleta.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] Definir Secure do cookie TFAE por política de produção, não por VERCEL
Labels sugeridas: security, severity:medium

Descrição do problema
O cookie `tfae` usa `secure: !!process.env.VERCEL`. Isso é seguro no deployment Vercel atual, mas o projeto suporta execução self-hosted via `npm start`. Uma implantação HTTPS fora da Vercel pode entrar em produção com `VERCEL` ausente e criar cookie sem `Secure`.

Evidência
`server/app.mjs:164-171`

```js
.cookie("tfae", t, {
httpOnly: true,
secure: !!process.env.VERCEL,
sameSite: "strict",
...
})
```

Impacto
Em ambiente self-hosted vulnerável, a sessão staff pode ser transmitida por HTTP se a origem também responder sem TLS.

Sugestão de correção
Basear a flag em uma política explícita: `NODE_ENV === 'production'` ou `COOKIE_SECURE=true`, com default seguro e validação de startup em produção.

Critérios de aceite
- [ ] Todo ambiente de produção cria cookie TFAE com `Secure`.
- [ ] Startup falha ou alerta de forma bloqueante se produção estiver configurada sem cookie seguro.
- [ ] Teste automatizado cobre Vercel e self-hosted production.

- [ ] `httpOnly` e `SameSite=Strict` permanecem ativos.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Segurança] Reduzir detalhes expostos pelo health check público

Labels sugeridas: security, severity:low

Descrição do problema

`GET /api/health` retorna estado e latência do banco, estado do storage, presença de configuração staff e tempo de resposta. O erro textual de banco já é corretamente ocultado em produção, mas os demais sinais facilitam reconhecimento operacional.

Evidência

`server/app.mjs:79-109`

```js
res.status(ok ? 200 : 503).json({
ok,
database,
databaseLatencyMs,
storage,
staff: !!process.env.STAFF_PASSWORD_HASH,
responseTimeMs: Date.now() - started,
});
```

Impacto

Reconhecimento de dependências e detecção de cold start/degradação. Impacto baixo isoladamente; não há credenciais nem dados de cliente.

Sugestão de correção

Manter um liveness público mínimo (`{ ok: true }`) e mover readiness detalhado para endpoint protegido/observabilidade interna.

Critérios de aceite

- [ ] Health público não informa banco/storage/staff/latência detalhada.
- [ ] Readiness detalhado continua disponível para operação autenticada ou monitor interno.
- [ ] Nenhuma mensagem de exceção interna aparece em produção.

--- FIM ISSUE 6 ---

Fim do relatório. Base auditada: 6426c528bd3a2b1ef4cc86790a1a86b8cf338990. Gerador: docs/security-audit/generate_report.py.